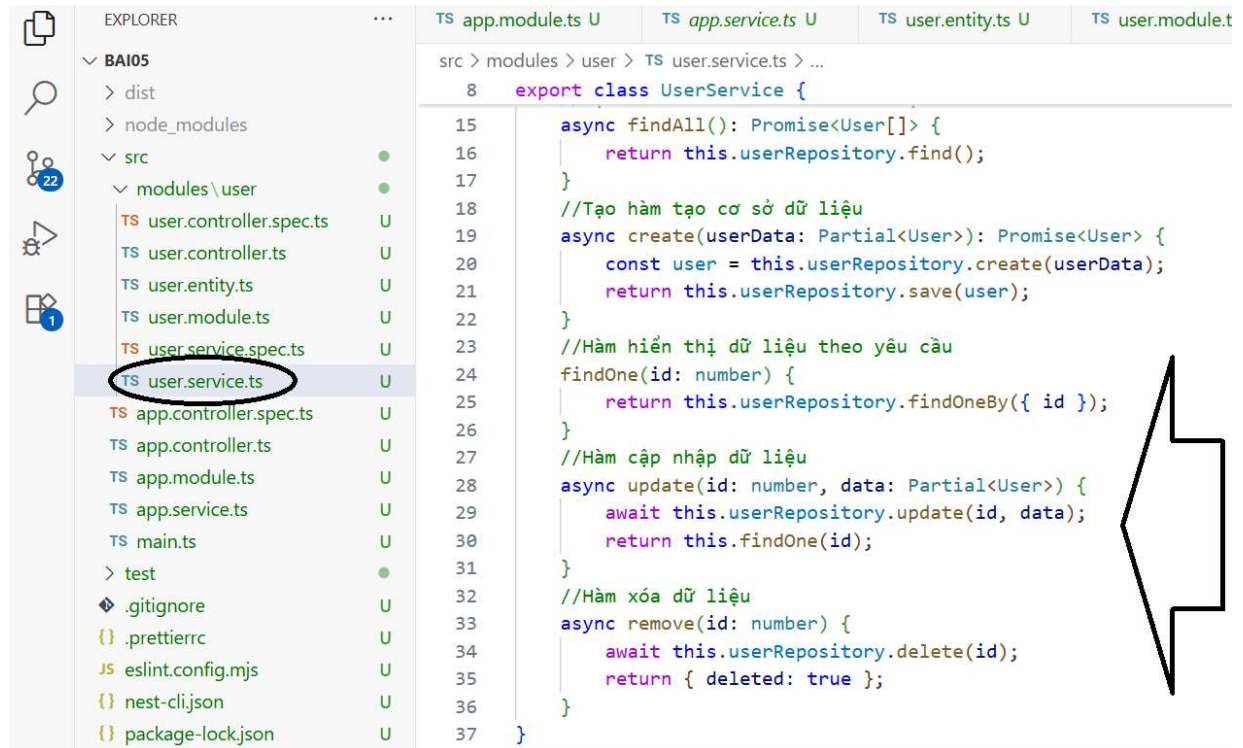


5.8. XÂY DỰNG CÁC HÀM XỬ LÝ DỮ LIỆU TIẾP THEO

- Bây giờ chúng ta sẽ tạo thêm 3 hàm xử lý dữ liệu nữa là: **Hiển thị dữ liệu theo yêu cầu** (ví dụ theo id), **Cập nhật dữ liệu** và **Xóa dữ liệu**.

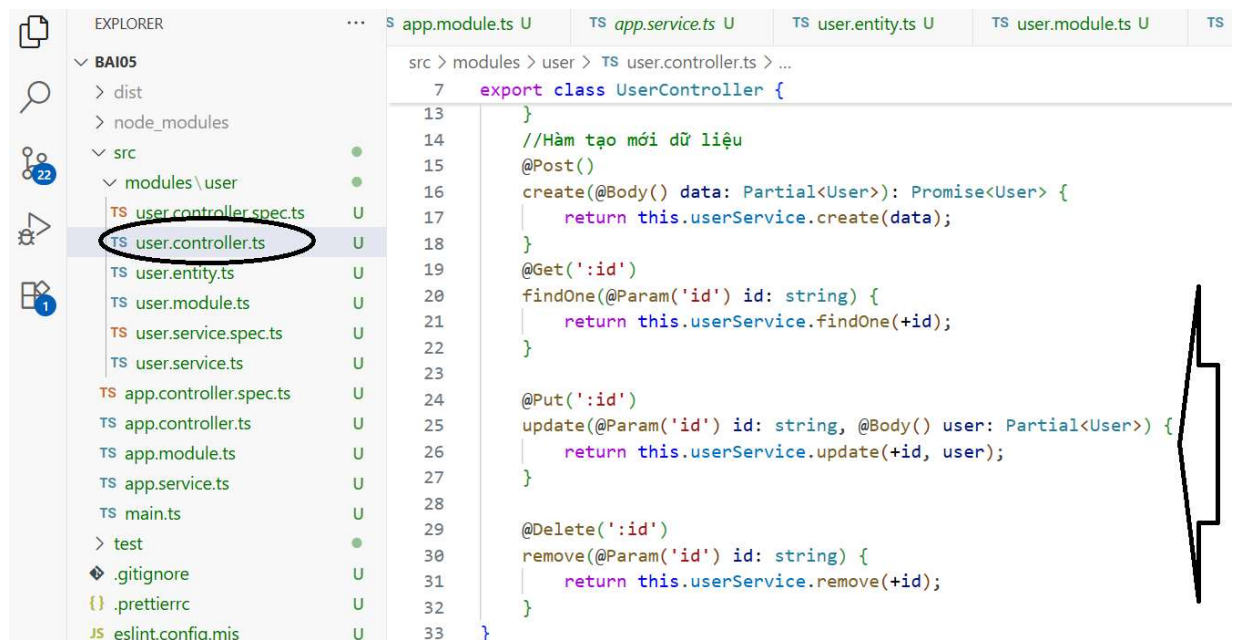
- Đầu tiên các bạn tạo **Service** cho 3 hàm xử lý bằng cách mở tập tin **users.service.ts** và viết thêm code cho 3 hàm như sau:



```

src > modules > user > TS user.service.ts > ...
8  export class UserService {
15
16      async findAll(): Promise<User[]> {
17          return this.userRepository.find();
18      }
19      //Tạo hàm tạo cơ sở dữ liệu
20      async create(userData: Partial<User>): Promise<User> {
21          const user = this.userRepository.create(userData);
22          return this.userRepository.save(user);
23      }
24      //Hàm hiển thị dữ liệu theo yêu cầu
25      findOne(id: number) {
26          return this.userRepository.findOneBy({ id });
27      }
28      //Hàm cập nhật dữ liệu
29      async update(id: number, data: Partial<User>) {
30          await this.userRepository.update(id, data);
31          return this.findOne(id);
32      }
33      //Hàm xóa dữ liệu
34      async remove(id: number) {
35          await this.userRepository.delete(id);
36          return { deleted: true };
37      }
    
```

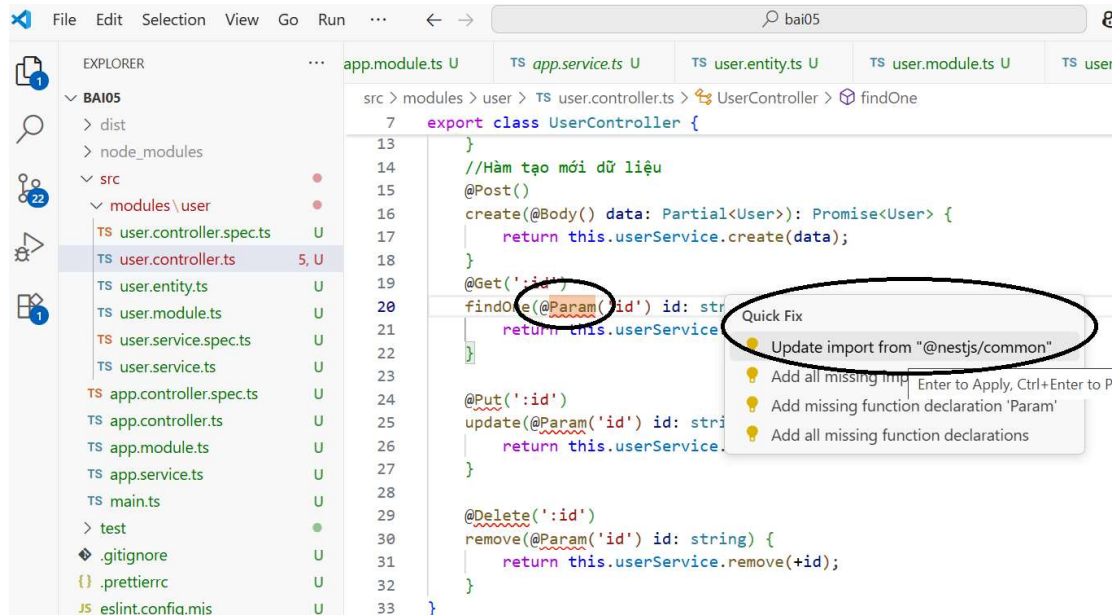
- Tiếp tục mở tập tin **user.module.ts** viết code để đưa 3 hàm vừa tạo vào **Controller**:



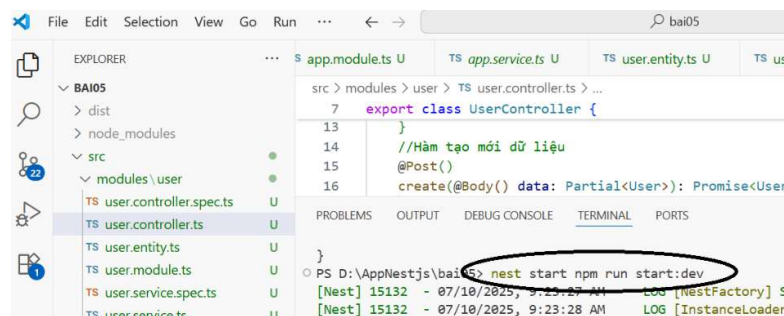
```

src > modules > user > TS user.controller.ts > ...
7  export class UserController {
13
14      //Hàm tạo mới dữ liệu
15      @Post()
16      create(@Body() data: Partial<User>): Promise<User> {
17          return this.userService.create(data);
18      }
19      @Get('/:id')
20      findOne(@Param('id') id: string) {
21          return this.userService.findOne(+id);
22      }
23
24      @Put('/:id')
25      update(@Param('id') id: string, @Body() user: Partial<User>) {
26          return this.userService.update(+id, user);
27      }
28
29      @Delete('/:id')
30      remove(@Param('id') id: string) {
31          return this.userService.remove(+id);
32      }
33
    
```

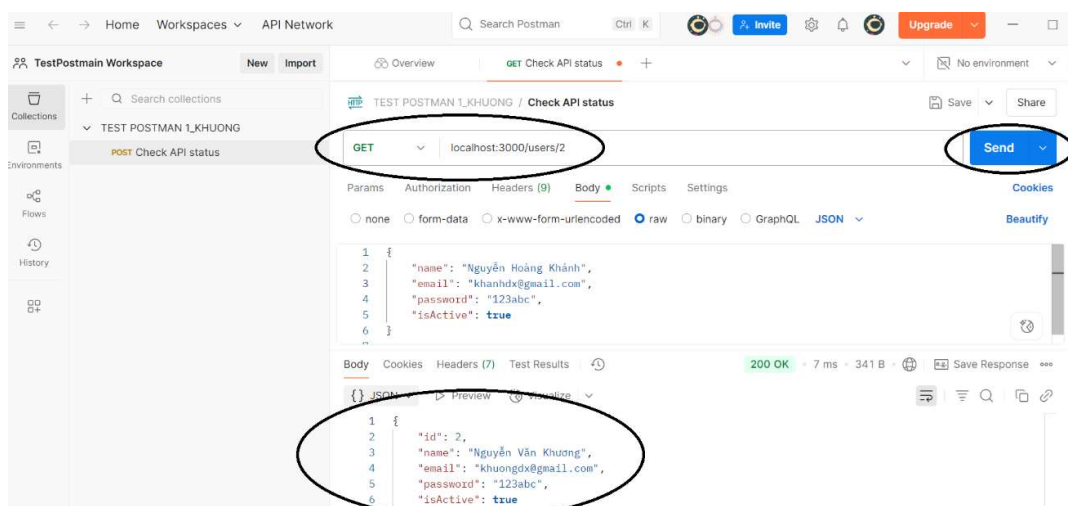
- **Lưu ý:** Nếu xuất hiện lỗi thì các bạn Click chuột phải vào chỗ lỗi, chọn **Quick fix**, và chọn **Update import...**



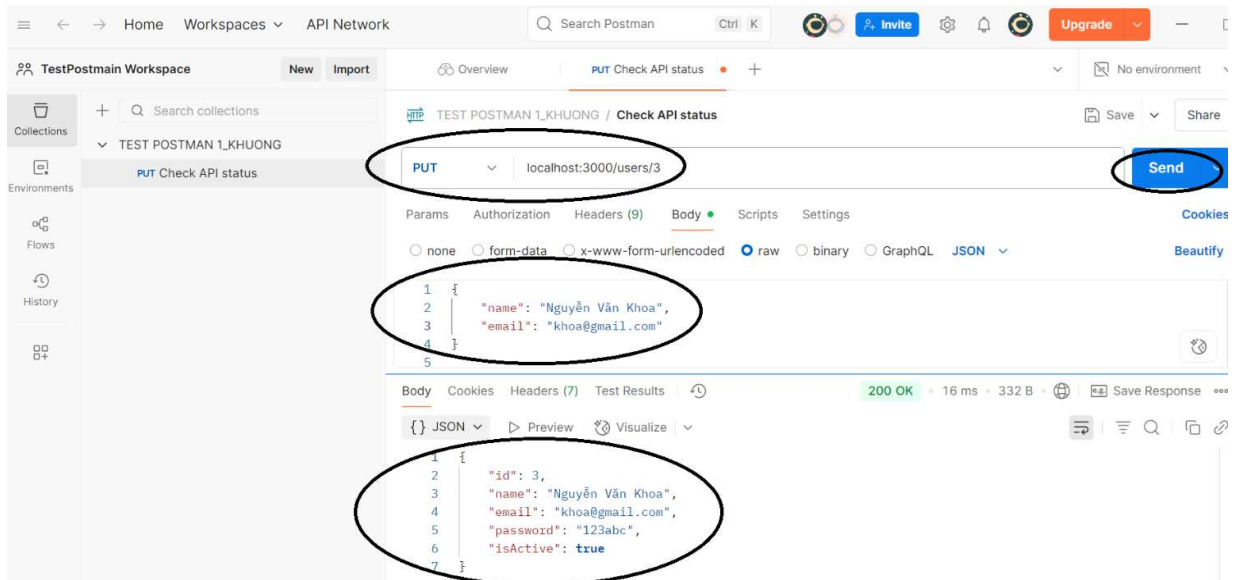
- Bây giờ các bạn biên dịch lại chương trình và mở **Postman** để kiểm tra các hàm vừa tạo:



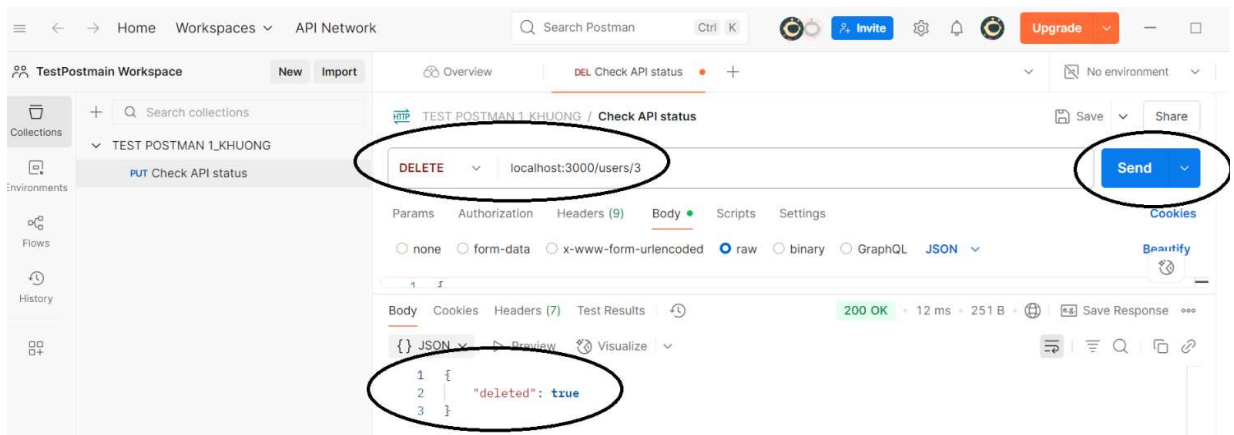
- Kiểm tra hàm **Hiện thị theo yêu cầu** trên **Postman**: Gõ đường dẫn như bên dưới với **id=2** (**localhost:3000/users/2**) thì sẽ hiển thị dữ liệu của người có **id=2**.



- Kiểm tra hàm **Cập nhật dữ liệu** trên **Postman**: Gõ đường dẫn như bên dưới với **id=3** để cập nhật người có **id=3**, chọn phương thức **PUT**, thay đổi thông tin trường **name** và **email** của người có thông tin **id=3**



- Kiểm tra hàm **Xóa dữ liệu** trên **Postman**: Gõ đường dẫn như bên dưới với **id=3** để xóa người có **id=3**, chọn phương thức **DELETE**.



- Như vậy các bạn đã xây dựng được 3 hàm tiếp theo để xử lý dữ liệu trong **Nestjs** là: Hiển thị theo yêu cầu, Cập nhật và xóa dữ liệu.